Day 3 Recap

What We Learned Today

KavinSchool

Day 3: What We Covered

Session 1: Conversation Context & Multi-turn Design

- The LLM API is stateless: your app manages history
- Context window strategies: truncation, summarization, sliding window
- Chatbot design: purpose, persona, conversation flow

Session 2: RAG Fundamentals

- Embeddings turn text into meaning-vectors
- Vector databases store and search embeddings
- Chunking strategies affect retrieval quality

Day 3: What We Covered

Session 3: Building a RAG-powered Chatbot

- End-to-end pipeline: load → chunk → embed → store → retrieve → generate
- Source citation: "answer ONLY from context" + [Source: filename]
- RAG vs fine-tuning vs prompt engineering: decision guide

Session 4: Integration & Channels

- Flask API wrapping for HTTP access
- Caching, shortcuts, and error handling
- Platform integration patterns (Slack, Teams)

Day 4 Preview: Production Best Practices & Capstone

Tomorrow we make it **production-ready**:

- **Security:** Prompt injection attacks and defenses
- **Cost & Latency:** Token economics, caching, optimization
- **Ethics & Bias:** Detection, mitigation, responsible use
- **Monitoring:** Logging, evaluation harness, drift detection
- **Capstone:** Build your own AI-powered developer assistant

> **Homework (optional):** Add 3 more documents to your knowledge base. Does retrieval quality improve or degrade? Why?

Quick Knowledge Check

1. Why is the LLM API **stateless** and what does your code do about it?
2. Name **three** context window management strategies.
3. What are the **two pipelines** in a RAG system?
4. What does an **embedding** represent?
5. Why is **chunk size** important for RAG quality?
6. What is the key prompt instruction that **prevents hallucination** in RAG?

Thank You!

Day 3 Complete

See you tomorrow for **Production Best Practices & Capstone!**

KavinSchool